11.4 The Nightingale System: A Case Study in Applying the ATAM

This section will describe the ATAM in practice using a case study based on an actual evaluation. Identifying information has been changed to protect the client's confidentiality.

PHASE 0: PARTNERSHIP AND PREPARATION

The client or organization for the evaluation, which had approached us after reading about the ATAM on our Web site, was a major producer of health care systems software, aimed at the hospital, clinic, and HMO markets. The system under consideration was called Nightingale. We learned that it was a large system expected to comprise several million lines of code and that it was well into implementation. Nightingale already had its first customer, a hospital chain with forty-some hospitals throughout the southwestern United States.

Why, we wanted to know, was our client interested in an architecture evaluation when the system was already well on its way to being fielded and sold? There were two reasons. First, if the architecture was fundamentally flawed in any way, it was much better to discover it sooner rather than later; second, the organization had strong ambitions to sell the system to many other customers, but recognized that it would have to tailor it specifically to the needs, applications, and regulatory environments of each one. Hence, while the architecture might be adequate for the first, kickoff customer, the client wanted to make sure that it was sufficiently robust and modifiable to serve as the basis for an entire product family of health care management systems.

The system would serve as the information backbone for the health care institutions in which it was installed. It would provide data about patients' treatment history as well as track their insurance and other payments. And it would provide a data-warehousing capability to help spot trends (such as predictors for relapses of certain diseases). The system would produce a large number of on- demand and periodic reports, each tailored to the institution's specific needs. For those patients making payments on their own, it would manage the work flow associated with initiating and servicing what amounts to a loan throughout its entire life. Further, since the system would either run (or at least be accessible) at all of the health care institution's facilities, it had to be able to respond to a specific office's configuration needs. Different offices might run different hardware configurations, for instance, or require different reports. A user might travel from one site to another, and the system would have to recognize that user and his or her specific information needs, no matter the location.

Negotiations to sign a statement of work took about a month-par for the course when legalities between two large organizations are involved-and when it was complete we formed an evaluation team of six people,[2] assigning roles as shown in Table 11.4.

[2] Six is a large team. As we mentioned earlier, teams are usually three to five people and four is average. In this case, two of the team members were new to the ATAM process and were added to give them experience.

Table 11.4. Evaluation Team Role Assignments

Member	Role
1	Team leader, evaluation leader, questioner
2	Evaluation leader, questioner
3	Timekeeper, questioner
4	Scenario scribe, questioner, data gatherer
5	Questioner, process enforcer
6	Proceedings scribe, process observer

For this exercise, we assigned two evaluation leaders who would take turns facilitating the proceedings. We have found this scheme markedly helpful in reducing fatigue and stress, and it makes for better results. We chose our questioners based on their familiarity with performance and modifiability. We also chose people with experience in integrating COTS products, since our client told us early on that Nightingale employed a few dozen commercial software packages. Happily, one of our questioners also had experience working in the health care industry.

We held a one-day kickoff meeting attended by the evaluation team, the project manager, the lead architect, and the project manager for Nightingale's first customer. The last three constituted the decision makers for Nightingale. At the meeting, we heard more about Nightingale's capabilities and requirements, received a catalog of available architectural documentation (from which we chose those we wanted to examine), and compiled a list of stakeholders to attend phase 2. We agreed on a schedule for the phase 1 and phase 2 meetings and for the delivery of the final report. Finally, we went over the presentations that the project manager and the architect, respectively, would be requested to make for steps 2 and 3 of phase 1, and made sure they were clear on the information we would want to see.

Later, before phase 1, our team met for two hours. The team leader went over the role assignments once again and made sure everyone knew his or her duties. Also, we walked through the architecture documentation we had received, making note of the patterns and tactics it indicated. This pre-meeting helped the team arrive at the evaluation somewhat knowledgeable about the architecture (thus increasing everyone's confidence), and it laid the groundwork for step 4, in which patterns and approaches would be cataloged.

In the Nightingale evaluation, the meeting also raised a red flag about the documentation, which was incomplete and unclear. Whole sections had not yet been written, and by and large the architecture was presented as a set of inadequately defined box-and-line diagrams. We felt that, were we to begin phase 1 at this point, we would not be on a firm conceptual footing. So we telephoned the architect and asked him to verbally fill in some of the blanks. Then, though we knew there were still gaps in our knowledge, at least we felt comfortable enough to begin the evaluation. We made a note that inadequate documentation was a risk that we needed to catalog.

PHASE 1: EVALUATION

As called for in phase 1, the evaluation team met with the project's decision makers. In addition to those who had attended the kickoff meeting (the project manager, the lead architect, and the project manager for Nightingale's kickoff customer), two lead designers participated.

Step 1: Present ATAM

The evaluation leader used our organization's standard viewgraph package that explains the method. The hour-long presentation lays out the method's steps and phases, describes the conceptual foundations underlying the ATAM (such as scenarios, architectural approaches, sensitivity points, and the like), and lists the outputs that will be produced by the end of the exercise.

The decision makers were already largely familiar with ATAM, having heard it described during the phase 0 discussions, so this step proceeded without a hitch.

Step 2: Present Business Drivers

At the evaluation, the project manager for the client organization presented the business objectives for the Nightingale system from the development organization, as well as from organizations they hoped would be customers for the system. For the development organization, Nightingale addressed business requirements that included

- support for their kickoff customer's diverse uses (e.g., treatment tracking, payment histories, trend spotting, etc.).
- creation of a new version of the system (e.g., to manage doctors' offices) that the development organization could market to customers other than the kickoff customer.

The second business driver alerted us to the fact that this architecture was intended for an entire software product line (see Chapter 14), not just one system.

For the kickoff customer, Nightingale was to replace the multiple existing legacy systems, which were

- old (one was more than 25 years old).
- n based on aging languages and technology (e.g., COBOL and IBM assembler).
- difficult to maintain.
- unresponsive to the current and projected business needs of the health care sites.

The kickoff customer's business requirements included

- the ability to deal with diverse cultural and regional differences.
- the ability to deal with multiple languages (especially English and Spanish) and currencies (especially the U.S. dollar and Mexican peso).
- a new system at least as fast as any legacy system being replaced.
- a new single system combining distinct legacy financial management systems.

The business constraints for the system included

- a commitment to employees of no lost jobs via retraining of existing employees.
- the adoption of a "buy rather than build" approach to software.
- recognition that the customer's marketplace (i.e., number of competitors) had shrunk.

The technical constraints for the system included

- use of off-the-shelf software components whenever possible.
- a two-year time frame to implement the system with the replacement of physical hardware occurring every 26 weeks.

The following quality attributes were identified as high priority:

- *Performance.* Health care systems require quick response times to be considered useful. The 5-second transaction response time of the legacy system was too slow, as were the legacy response times for online queries and reports. System throughput was also a performance concern.
- *Usability.* There was a high turnover of users of the system, so retraining was an important customer issue. The new system had to be easy to learn and use.

- *Maintainability.* The system had to be maintainable, configurable, and extensible to support new markets (e.g., managing doctors' offices), new customer requirements, changes in state laws and regulations, and the needs of the different regions and cultures.

The manager identified the following quality attributes as important, but of somewhat lower priority:

- *Security.* The system had to provide the normal commercial level of security (e.g., confidentiality and data integrity) required by financial systems.
- *Availability.* The system had to be highly available during normal business hours.
- *Scalability.* The system had to scale up to meet the needs of the largest hospital customers and down to meets the needs of the smallest walk-in clinics.
- *Modularity.* The developing organization was entertaining the possibility of selling not just new versions of Nightingale but individual components of it. Providing this capability required qualities closely related to maintainability and scalability.
- *Testability and supportability.* The system had to be understandable by the customer's technical staff since employee training and retention was an issue.

Step 3: Present Architecture

During the evaluation team's interactions with the architect, before as well as during the evaluation exercise, several views of the architecture and the architectural approaches emerged. Key insights included the following:

- Nightingale consisted of two major subsystems: OnLine Transaction Manager (OLTM) and Decision Support and Report Generation Manager (DSRGM). OLTM carries interactive performance requirements, whereas DSRGM is more of a batch processing system whose tasks are initiated periodically.
- Nightingale was built to be highly configurable.
- The OnLine Transaction Manager subsystem was strongly layered.
- Nightingale was a repository-based system; a large commercial database lay at its heart.
- Nightingale relied heavily on COTS software, including the central database, a rules engine, a work flow engine, CORBA, a Web engine, a software distribution tool, and many others.
- Nightingale was heavily object oriented, relying on object frameworks to achieve much of its configurability.

Figure 11.3 shows a layered view of OLTM rendered in the informal notation used by the architect. Figure 11.4 depicts how OLTM works at runtime by showing the major communication and data flow paths among the parts of the system deployed on various hardware processors. We present these figures basically as we were given them to give you a better understanding of the reality of an ATAM evaluation. Note that they do not cleanly map; that is, in Figure 11.3 there is a transaction manager and CORBA, but these do not occur in Figure 11.4. This type of omission is typical of many of our ATAM evaluations, and one of the activities that occurs during step 3 is that the evaluators ask questions about the inconsistencies in the diagrams in an attempt to come to some level of understanding of the architecture. Figure 11.5 shows a similar runtime view of OLTM in which a transaction can be traced throughout the system, again with similar inconsistencies and, in this case, without a description of the meaning of the arrows. We determined that these arrows also represented data flow.

Figure 11.3. Layered view of the OLTM in the the architect's informal notation

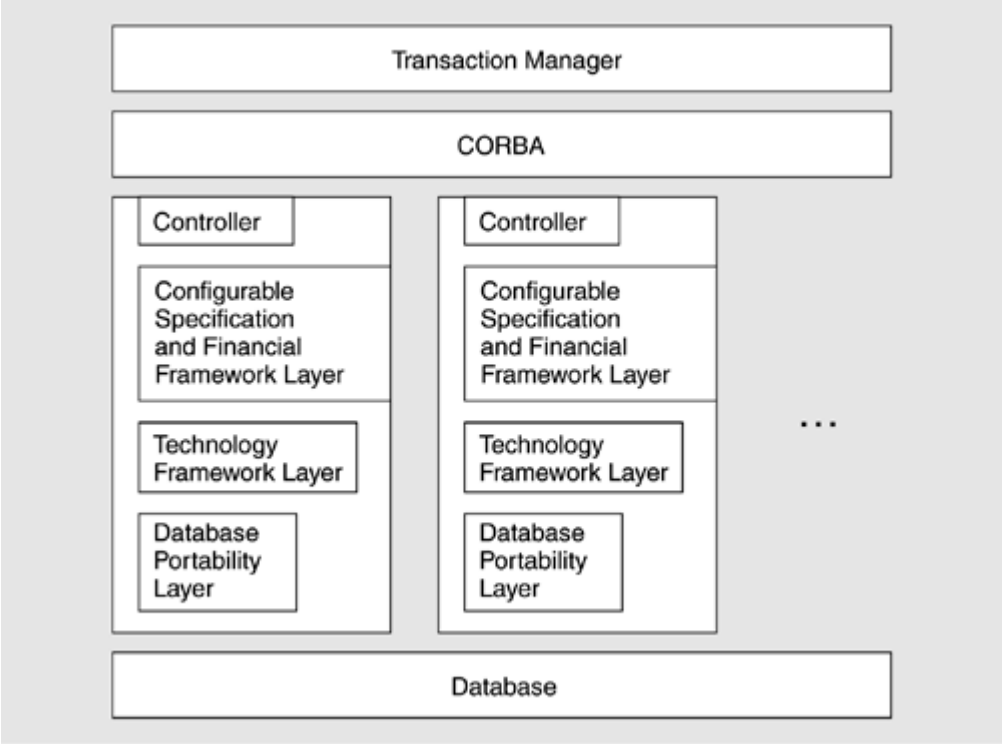


Figure 11.4. A view showing communication, data flow, and processors of the OLTM

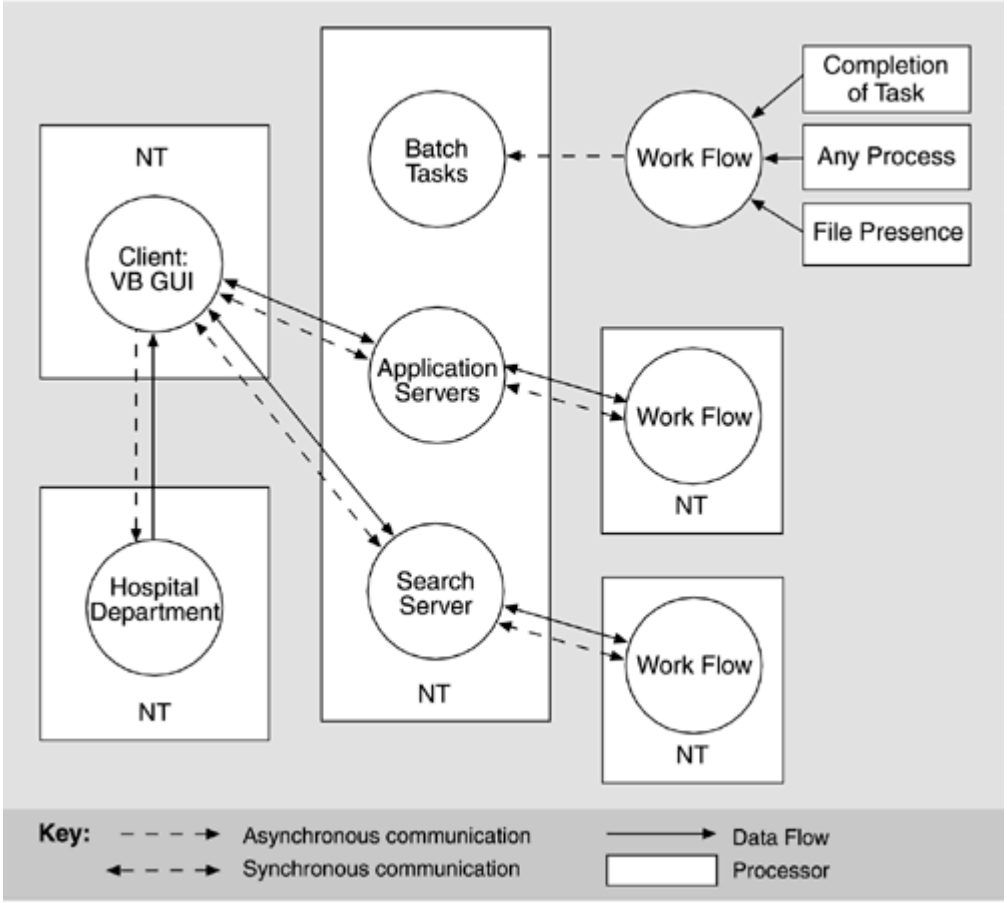
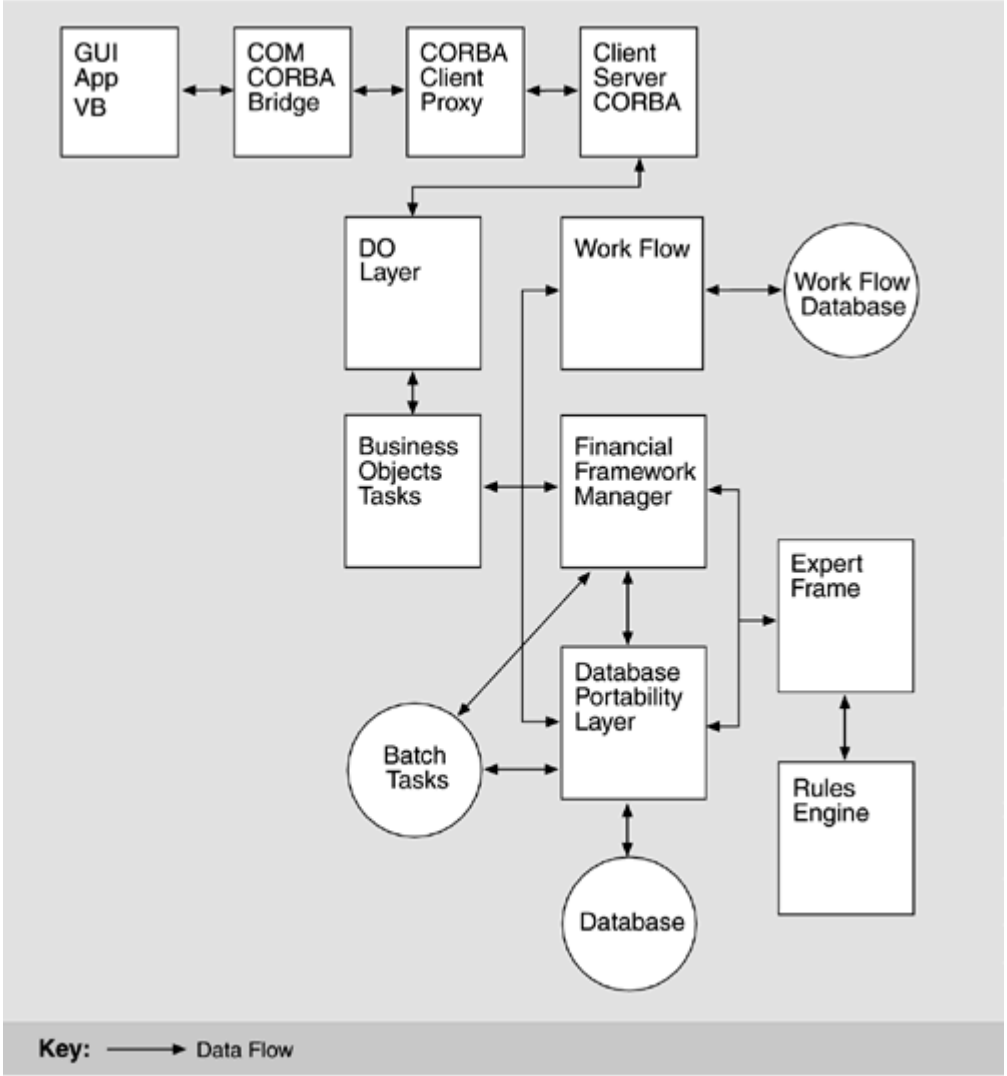


Figure 11.5. Data flow architectural view of the OLTM



All of these views of the Nightingale are equally legitimate and carry important information. Each shows an aspect relevant to different concerns, and all were used to carry out the analysis steps of the ATAM exercise.

Step 4-Catalog Architectural Approaches

After the architecture presentation, the evaluation team listed the architectural approaches they had heard, plus those they had learned about during their pre-evaluation review of the documentation. The main ones included

- layering, especially in OLTM.
- object orientation.
- use of configuration files to achieve modifiability without recoding or recompiling.
- client-server transaction processing.
- a data-centric architectural pattern, with a large commercial database at its heart.

These and other approaches gave the evaluation team a conceptual footing from which to begin asking probing questions when scenario analysis began.

Step 5-Generate Quality Attribute Utility Tree

Table 11.5 shows the utility tree generated during the Nightingale ATAM exercise. Notice that all of the quality attributes identified during step 2 appear and that each is refined into one or more specific meanings.

A few of the quality attribute refinements have no scenarios associated with them. That often happens and it is not a problem. People are sometimes able to think of a reasonable-sounding refinement for a quality attribute, but, when pressed to instantiate it in the context of their own system, discover that it does not really apply.

To capture the utility tree for all to see, the proceedings scribe used a flipchart page for each quality attribute and taped it to the wall. Then, as that quality attribute was refined and instantiated with scenarios, she captured the information on that flipchart or on continuation flipcharts taped underneath.[3]

[3] We have also experimented with capturing the utility tree online in a table like Table 11.5 and projecting it directly from the computer. This makes the tree easier to build and modify but the participants can see only one screen's worth at any time. Seeing the whole utility tree helps stimulate thinking and identify gaps. Collaborative-work software systems would seem to be ideal here, but it is hard to beat flipcharts and masking tape for simplicity, reliability, and economy.

The scenarios in Table 11.5 are annotated with the priority rankings assigned by the decision makers present. The first of each ordered pair indicates the importance of the capability; the second indicates the architect's estimation of the difficulty in achieving it.

Table 11.5. Tabular Form of the Utility Tree for the Nightingale ATAM Exercise

Quality Attribute	Attribute Refinement	Scenarios
Performance	Transaction response time	A user updates a patient's account in response to a change-of-address notification while the system is under peak load, and the transaction completes in less than 0.75 second. (H,M)
		A user updates a patient's account in response to a change-of-address notification while the system is under twice the current peak load, and the transaction completes in less than 4 seconds. (L,M)
	Throughput	At peak load, the system is able to complete 150 normalized transactions per second. (M,M)
	Generating reports	No scenarios suggested.
Usability	Proficiency training	A new hire with two or more years experience in the business becomes proficient in Nightingale's core functions in less than 1 week. (M,L)
		A user in a particular context asks for help, and the system provides help for that context. (H,L)
	Normal operations	A hospital payment officer initiates a payment plan for a patient while interacting with that patient and completes the process without the system introducing delays. (M,M)
Configurability		A hospital increases the fee for a particular service. The configuration team makes the change in 1 working day; no source code needs to change. (H,L)
Maintainability		A maintainer encounters search- and response-time deficiencies, fixes the bug, and distributes the bug fix. (H,M)
		A reporting requirement requires a change to the report-generating metadata. (M,L)
		The database vendor releases a new version that must be installed in a minimum amount of time. (H,M)
Extensibility	Adding new product	A product that tracks blood bank donors is created. (M,M)
Security	Confidentiality	A physical therapist is allowed to see the part of a patient's record dealing with orthopedic treatment, but not other parts nor any financial information. (H,M)
	Integrity	The system resists unauthorized intrusion. (H,M)
Availability		The database vendor releases new software, which is hot-swapped into place. (H,L)
		The system supports 24/7 Web-based account access by patients. (L,L)
Scalability	Growing the system	The kickoff customer purchases a health care company three times its size, requiring a partitioning of the database. (L,H)
		The kickoff customer divests a business unit. (L,M)
		The kickoff customer consolidates two business units. (L,M)
		The developing organization wants to sell components of Nightingale. (M,L)
Modularity	Functional subsets	Build a system that can function autonomously with core functionality. (M,L)
	Flexibility to replace COTS products	Replace the commercial database with one by another vendor. (H,M)
		Replace the operating system. (H,M)
		Replace the database portability layer. (H,M)
		Replace the transaction manager. (H,M)
		Replace the work flow engine. (H,M)
		Replace the commercial accounting package. (H,M)
		Replace Solaris on the Sun platforms that host the database. (H,M)
		Replace the rules engine. (H,M)

Quality Attribute	Attribute Refinement	Scenarios
Interoperability		Build a system that interfaces with the epidemiological database at the National Centers for Disease Control. (M,M)
Testability		
Supportability		

Notice that some of the scenarios are well formed according to our earlier discussion, others have no stimulus, and still others have no responses. At this stage, the imprecision in scenario specification is permissible as long as the stakeholders understand the meaning. If the scenarios are selected for analysis, then the stimulus and response must be made explicit.

Step 6-Analyze Architectural Approaches

The utility tree exercise produced no scenarios ranked (H,H), which indicates high-importance, high-difficulty scenarios that merit high analytical priority. So we looked for (H,M) scenarios, a cluster of which appeared under "Modularity," hypothesizing the replacement of various COTS products in the system. Although extensive use of COTS was a purposeful strategy to reduce development risk, it was also worrisome to the project's management because it was felt that the system (and the customers to whom it was sold) would be at the mercy of a large number of COTS vendors. Therefore, achieving architectural flexibility to swap out COTS products was of keen interest.

We walked through each of the scenarios with the architect. Each consumed, on average, about a half hour.[4] Since these were scenarios about changes, we asked about the range and impact of the changes. We learned the following.

[4] .In evaluation after evaluation, the first scenario analyzed invariably takes the most time, perhaps as much as three times the average.

- Replacing the commercial database with a database supplied by another vendor would be difficult. A dialect of SQL (a superset of ANSI-standard SQL) specific to the current database vendor was used throughout Nightingale, as were several vendor-specific tools and components. The architect considered replacing the database as highly unlikely and so was not concerned that shifting to another system would be very expensive. This was news to the project manager, however, who was not so sure that the scenario was out of the question. We recorded our first analysis-based architectural risk: "Because Nightingale uses vendor-specific tools, components, and an SQL dialect not supported by or compatible with databases supplied by other vendors, replacing the database would be extremely difficult and expensive, requiring several staff-years of effort." The architectural decision to wed the architecture to the database was also recorded as a sensitivity point, negatively affecting modifiability.

- Replacing one operating system with another would be a reasonably straightforward change. On the server side, the operating system was insulated by a layer, which would confine the necessary changes to a small portion. However, OLTM relies on NT authentication facilities directly, and a replacement operating system would have to provide something similar for the change to be straightforward. On the DSRGM side, all operating system dependencies had already been eliminated in the source code; DSRGM was developed on a Windows NT platform but deployed on UNIX, providing compelling evidence that it was already independent of the operating system. Here we recorded our first nonrisk: "Because operating system dependencies have been localized or eliminated from OLTM and DSRGM, replacing the operating system with another one would require only a small modification." Encapsulating operating system dependencies was recorded as a sensitivity point, positively affecting modifiability.

- Changing the rules engine raised several issues of concern. This scenario was not a farfetched one, because we learned that there were associated performance and maintainability concerns associated with using the rules engine. The likely scenario would be to remove, not replace, the rules engine and then implement the rules directly in C++. Since forward chaining among the rules had been disallowed (specifically-and wisely-to keep this option open), the rules were effectively procedural and could be compiled. Such a change would have several serious effects:

  - It would likely improve performance (although this question had not yet been answered authoritatively).

  - It would obviate the need for personnel trained in the rules language and knowledgeable about the rules engine.

  - It would deprive the development team of a useful rules development and simulation environment.

  - It would lead to the possibility that the rules could become "buried" in the rest of the C++ code and make it easier for them to become entangled in functional code not strictly related to rules, and hence harder to recognize and maintain.

  - It would remove the possibility that the rules could reference some object that in fact did not exist, a possibility that exists today and represents an error that could conceivably survive past testing and into a production system. Writing the rules in C++ would eliminate this error at compile time.

  To facilitate this change, a rule-to-C++ code generator would need to be written, a development effort of significant scope and unknown difficulty. For this scenario, we recorded as a risk the major effort needed to remove the rules engine. We also recorded using a rules engine (as opposed to C++ code) as a tradeoff point in the architecture. This made development easier and changes to the rule base easier; however, these benefits came at the cost of decreased performance, specially trained developers, and more difficult testing.

And so forth. We continued this scenario, investigating replacement of the commercial Web-hosting engine, the commercial accounting package, the work flow engine, and the Solaris operating system on the Sun platforms.

At this point, the phase 1 meeting ended. We had recorded six sensitivity points, one tradeoff point, four risks, and five nonrisks.

PHASE 2: EVALUATION (CONTINUED)

The phase 2 meeting commenced after a hiatus of two weeks. During the break, the evaluation team wrote up those parts of the final report that could be completed: the business drivers, the presented architecture, the list of approaches, the utility tree, and the phase 1

analysis. We also interacted via telephone with the architect to check our understanding of some technical points, and with the project manager to make sure that a good stakeholder representation would be present for phase 2.

For phase 2, we had nine stakeholders present in addition to the project decision makers present during phase 1. They included developers, maintainers, representatives from the kickoff customer, and two end users.

The first activities of phase 2 were to repeat step 1 (describing the ATAM) for the new participants, and then recap the results of phase 1 to bring everyone up to speed. After that, steps 7, 8, and 9 were carried out.

Step 7-Brainstorm and Prioritize Scenarios

The stakeholders were a productive group, contributing a total of 72 scenarios during this step. More than a dozen of those scenarios were found at the leaves of step 5's utility tree but were not analyzed during phase 1. This was not only proper but encouraged. In this way, the stakeholders were expressing the view that some scenarios deserved more attention than they had received during phase 1.

Table 11.6 contains a selection of some of the more interesting scenarios that emerged during step 7. Notice that many of them are not particularly well structured, and some are downright cryptic. This reflects the spontaneous nature of a brainstorming exercise in which everyone is actively engaged. Rather than spend several minutes structuring and wordsmithing each scenario as it arises, we like to concentrate on capturing thoughts while they are fresh in people's minds. If a scenario's meaning needs to be polished before voting occurs or before it is analyzed, then we are happy to spend the necessary time doing so (with the help of the person who proposed it).

Table 11.6. Brainstormed Scenarios

Number	Scenario
1	Previously public data is made private, and access is adjusted accordingly.
2	Data in the information hub is replicated to a branch clinic, and performance is degraded.
3	A rule in the rule engine fires, and data access is too slow.
4	A user posts a patient's payment at a busy time, and response is slow (in a testing environment).
5	A user in one business unit needs to perform actions on behalf of other business units.
6	Decide to support German.
7	Add an epidemiologist role and supporting functionality.
8	Sell Nightingale to a five-person doctor's office and have it support their business.
9	A user requests a new field for asynchronous queries.
10	In response to a complaint, a hospital discovers it has been incorrectly charging for bedpans for six months.
11	A hospital needs to centralize the record maintenance process across multiple affiliates; associated business process is re-engineered.
12	A manager wants a report on historical payment delinquency rates for people who were treated for cuts and lacerations.
13	"What-if" scenario: A proposed law change is applied to an account.
14	A defect corrupts data and is not detected until the next reporting cycle.
15	Nightingale is installed in a hospital, and the hospital's existing database must be converted.
16	An error in the replication process causes a transaction database to be out of sync with the backup database.
17	An error in the system causes all payments to accounts in Arizona to be unpostable.
18	A transaction log audit trail fails for three days (how to recover?).
19	An affiliate redefines a business day and month.
20	Receive payment post information from an insurance company's database system, given its metadata definition.
21	Introduce a new work flow process for patient check-in and check-out.
22	Batch processes are initiated based on time and events.
23	Main communication to branch clinics from the information hub goes down.
24	A branch clinic database server fails to boot.
25	A report needs to be generated using information from two hospitals that use different configurations.
26	A remittance center submits the same batch of payments twice, and activity occurs after the second submission.
27	A rehabilitation therapist is assigned to another hospital, but needs read-only access to the treatment histories of his or her former patients.
28	Distribute a set of changes to a set of health care sites consistently (forms and configurations).
29	A fire in the data center forces the information hub to be moved to a new location.
30	One hospital sells a large number of accounts payable to another business unit.

Number	Scenario
31	Change the rules for generating a warning about conflicting medications.
32	A user in a hospital's finance office wants to change output from paper to online viewing.
33	The phone company changes an area code.
34	A malicious account administrator has slowly transferred small amounts into various accounts of his friends. How to discover and determine extent?

After merging a few almost-alike scenarios, the stakeholders voted. We assigned 22 votes to each stakeholder (72 scenarios times 30%, rounded up to the nearest even integer), which they cast in two passes. We tallied the votes and spent a half-hour with the group placing the dozen or so highest-priority scenarios in the utility tree created during step 5. For this exercise, all of the high-priority step 7 scenarios were straightforwardly placed as new leaves of existing branches in the utility tree. This suggested that the architect was thinking along the same lines as the stakeholders in terms of important quality attributes.

After reconciling the new scenarios with the utility tree, we began analyzing the scenarios that received the most votes.

Step 8-Analyze Architectural Approaches

During step 8, we analyzed seven additional scenarios, a number slightly above average for an ATAM exercise. In deference to space limitations, the Scenario 15 sidebar summarizes the analysis for just one of them.

Step 9-Present Results

Step 9 is a one- to two-hour presentation summarizing the results and findings of the exercise. It begins with a boilerplate set of slides that contains a method recap and blank template slides that can be filled in with the business drivers summary, the architecture summary, the list of approaches, the utility tree, the scenario analysis, and the list of analysis outputs.

The evaluation team meets during the evenings of phase 2 to compile all the results gathered so far. The phase 2 agenda also contains a block of time before step 9 when the team can caucus and complete the package.

In addition to the risks, nonrisks, sensitivity points, and tradeoff points, the team presents risk themes that seem to systematically underlie the problematic areas of the architecture, if any. This is the only part of the results that the participants will not have already seen (and, for that matter, helped to identify). For each one, we also state why it matters in terms that will be meaningful to the client: We identify the stated business drivers that each risk theme jeopardizes.

Scenario 15: Nightingale is installed in a hospital and the hospital's existing database must be converted.

Not surprisingly, the architect had given this scenario a lot of thought, since carrying it out successfully was essential to the success of Nightingale. There was a documented procedure in place, which the architect drew for us on the whiteboard.

It often happens that a scenario leads to a deeper understanding of the architecture than was present before. Here, the architect had the information, but (reasonably) did not include it in the step 3 presentation, considering it ancillary.

Walking through the migration process convinced the evaluation team that a well-thought-out procedure was in place, with known strengths and reasonable limitations. It did not surprise us that the architect did not mention the process during his presentation of step 3. What did surprise us was that we saw nothing about it in the documentation package we received and reviewed prior to phase 1. When pressed about this, the architect admitted that the procedure was not yet documented, which we recorded as a risk. Offsetting this risk, however, was a nonrisk that we recorded: "The architecture supports a straightforward and effective data conversion and migration facility to support Nightingale installation."

For Nightingale, we identified three risk themes:

1. *Over-reliance on specific COTS products.* Here we cited the difficulties in swapping out the database, in removing the rules engine, and in relying on an old and possibly no-longer-supported version of the database portability layer. This risk theme threatened the business driver of a system that is maintainable.
2. *Error recovery processes were not fully defined. The customer's knowledge of available tools was incomplete.* Several scenarios dealt with discovering errors in the database and backing them out. While the architecture supported those procedures well enough, it was clear that the architects and designers were thinking about some of them for the first time. The representatives of the kickoff customer reported that they had no procedures in place (either of their own or inherited from the developing organization) for making such error corrections. This risk theme threatened the business driver of usability and support for the customer's enterprise.
3. *Documentation issues.* The state of documentation on the Nightingale project was inadequate. The team began to realize this as far back as the pre-phase 1 meeting, and several scenarios analyzed during phase 2 reinforced this opinion. While a large volume of detailed documentation (such as that produced via UML and the Rose model) existed, there was almost no introductory or overview documentation of the architecture, which is critical for training, adding people to the project, maintenance, and guiding development and testing. The extensive rule base that governed the behavior of Nightingale was undocumented, as was the data conversion and migration procedure. Lacking such documentation, the system would be unmaintainable by the kickoff customer, who was on the verge of inheriting it, thus jeopardizing one of the key business drivers for Nightingale-support for the customer's enterprise.

PHASE 3: FOLLOW-UP

The tangible output of the ATAM is a final report that contains a list of risks, nonrisks, sensitivity points, and tradeoff points. It also contains a catalog of architectural approaches used, the utility tree and brainstormed scenarios, and the record of analysis of each selected scenario. Finally, the final report contains the set of risk themes identified by the evaluation team and an indication of which business drivers are jeopardized by each one.

Like the presentation of results, we use a boilerplate template that has many of the standard sections (such as a description of the ATAM) completed and templates for other sections ready to be filled in. We also write some of the final report-for instance, the utility tree and step 6 analysis-during the hiatus between phases 1 and 2. Preparation pays off; whereas it used to take about two weeks to produce a final report for an ATAM client, we can now produce a high-quality comprehensive report in about two days.

TABLE OF CONTENTS PREVIOUS NEXT